

GUÍA DE INSTALACIÓN Y CONFIGURACIÓN · UNIDAD DE APRENDIZAJE 2

Entorno de programación en C para Windows

gcc (MSYS2), variables de entorno, VS Code, Git Bash y GitHub

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Sistema operativo	Windows 10 u 11 de 64 bits
Tiempo estimado	60 a 90 minutos (depende de la velocidad de internet)
Persona facilitadora	Giovanni Antonio Coto Calderón

Qué va a instalar y para qué

- **MSYS2 con gcc (UCRT64):** el compilador que convierte su código en C en un programa ejecutable (.exe).
- **Visual Studio Code:** el editor donde escribirá, compilará y ejecutará sus programas.
- **Extensión C/C++ de Microsoft:** colores, autocompletado y detección de errores en VS Code.
- **Git para Windows (incluye Git Bash):** el control de versiones y la terminal que usaremos en el curso.
- **Cuenta de GitHub:** donde vive su portafolio de evidencias.

Orden	Paso	Sección
1	Preparar Windows (extensiones de archivo y carpeta de trabajo)	1
2	Instalar MSYS2 y el compilador gcc	2
3	Agregar gcc a la variable de entorno Path	3
4	Instalar Git para Windows	4
5	Instalar y configurar Visual Studio Code	5
6	Probar todo con un programa en C	6
7	Conectar con GitHub y el portafolio	7

Si trabaja en un equipo del laboratorio del INA, es posible que ya esté todo instalado: haga primero la prueba de la sección 6. Si funciona, solo necesita las secciones 5.3 a 5.5 y 7.

1. Preparar Windows

1.1 Mostrar las extensiones de los archivos

Así evitará archivos como hola.c.txt que gcc no reconoce.

1. Abra el **Explorador de archivos**.
2. Windows 11: menú **Ver** → **Mostrar** → **Extensiones de nombre de archivo**. Windows 10: pestaña **Vista** → marque **Extensiones de nombre de archivo**.

1.2 Crear la carpeta de trabajo

Use una ruta **corta, sin espacios, sin tildes y sin ñ**, por ejemplo **C:\INA**. Ahí clonará su repositorio del portafolio en la sección 7. Evite trabajar dentro de carpetas sincronizadas con OneDrive.

2. Instalar MSYS2 y el compilador gcc

MSYS2 es un entorno que trae para Windows las herramientas de compilación de GNU. Usaremos su versión **UCRT64**, la recomendada actualmente para Windows 10 y 11.

2.1 Descargar e instalar MSYS2

1. Entre a <https://www.msys2.org> y descargue el instalador (archivo **msys2-x86_64-...exe**).
2. Ejecútelo y deje la carpeta de instalación predeterminada: **C:\msys64**. No la cambie: el resto de la guía usa esa ruta.
3. Al terminar, deje marcada la opción para abrir MSYS2 y presione **Finalizar**. Se abrirá una terminal de MSYS2.

2.2 Instalar el compilador

En la terminal de MSYS2 escriba (o pegue con clic derecho) este comando y presione Enter:

Terminal de MSYS2

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

1. Cuando pregunte qué paquetes instalar (**Enter a selection (default=all)**), presione **Enter** para aceptar todos.
2. Cuando pregunte **Proceed with installation? [Y/n]**, escriba **Y** y presione Enter.
3. Espere a que termine la descarga e instalación (varios minutos). Cierre la terminal de MSYS2.

El paquete toolchain incluye gcc (compilador de C), g++ (C++), gdb (depurador) y make. Si en algún momento quiere actualizar MSYS2, abra la terminal MSYS2 UCRT64 y ejecute `pacman -Syu`.

3. Agregar gcc a la variable de entorno Path

La variable **Path** le dice a Windows en qué carpetas buscar programas. Al agregar la carpeta de gcc, podrá compilar desde cualquier terminal, incluida la de VS Code.

1. Presione la tecla **Windows**, escriba **variables de entorno** y elija **Editar las variables de entorno de esta cuenta**.
2. En **Variables de usuario**, seleccione **Path** y presione **Editar....**

3. Presione **Nuevo** y escriba exactamente: **C:\msys64\ucrt64\bin**
4. Presione **Aceptar** en todas las ventanas.
5. **Cierre todas las terminales y VS Code** (si estaban abiertos): solo las ventanas nuevas leen el Path actualizado.

3.1 Verificar

Abra una ventana nueva de **Símbolo del sistema** (cmd) o PowerShell y escriba:

Comando	¿Qué hace?	Resultado esperado
<code>gcc --version</code>	Muestra la versión del compilador.	gcc.exe (Rev..., Built by MSYS2 project) seguido del número de versión
<code>gdb --version</code>	Muestra la versión del depurador.	GNU gdb (GDB) ...
<code>where gcc</code>	Indica dónde está gcc.	C:\msys64\ucrt64\bin\gcc.exe

Si aparece “gcc no se reconoce como un comando...”

- Revise que la ruta en Path sea exactamente **C:\msys64\ucrt64\bin** (sin espacios ni barras de más).
- Confirme que existe el archivo **C:\msys64\ucrt64\bin\gcc.exe**; si no existe, repita la sección 2.2.
- Cierre y vuelva a abrir la terminal (o reinicie la sesión de Windows).

4. Instalar Git para Windows (incluye Git Bash)

1. Entre a <https://git-scm.com>, sección **Downloads** → **Windows**, y descargue el instalador de 64 bits.
2. Ejecútelo. Puede aceptar las opciones predeterminadas, con estos cambios recomendados:

Pantalla del instalador	Opción recomendada
Choosing the default editor	Use Visual Studio Code as Git's default editor (si VS Code aún no está instalado, deje Vim y cámbielo luego).
Adjusting the name of the initial branch	Override the default branch name → main .
Adjusting your PATH environment	Git from the command line and also from 3rd-party software (recomendado).
Credential helper	Git Credential Manager (permite iniciar sesión en GitHub desde el navegador).
Resto de pantallas	Valores predeterminados.

Verifique en una terminal nueva:

Comando	¿Qué hace?	Resultado esperado
<code>git --version</code>	Muestra la versión de git.	git version 2.x.x.windows.x

5. Instalar y configurar Visual Studio Code

5.1 Instalar

1. Entre a <https://code.visualstudio.com> y descargue la versión para Windows (**User Installer**, 64 bits).
2. En el instalador marque: **Agregar la acción “Abrir con Code”** (para archivos y carpetas) y **Agregar a PATH**.
3. Termine la instalación y abra VS Code.

5.2 Instalar las extensiones

Abra la vista de extensiones con **Ctrl + Shift + X** e instale:

Extensión	Publicada por	Para qué sirve
C/C++	Microsoft	Resaltado, autocompletado, detección de errores y depuración de C.
Spanish Language Pack (opcional)	Microsoft	Menús de VS Code en español.

5.3 Usar Git Bash como terminal

1. Presione **Ctrl + Shift + P**, escriba **Terminal: Select Default Profile** (en español: **Terminal: Seleccionar el perfil predeterminado**) y elija **Git Bash**.
2. Abra una terminal con **Ctrl + ñ** (o menú **Terminal → Nuevo terminal**). Debe ver un indicador terminado en \$.

En el curso los comandos se escriben en Git Bash: `ls`, `cd`, `mkdir -p`, `./programa.exe`. Es la misma sintaxis de Linux y de los servidores.

5.4 Abrir siempre la carpeta, no solo el archivo

Use **Archivo → Abrir carpeta...** y elija la carpeta del repositorio. Así VS Code lee la configuración de la plantilla del portafolio (carpeta `.vscode`).

5.5 Configuración incluida en la plantilla del portafolio

Su repositorio ya trae estos dos archivos; no necesita crearlos, pero conviene saber qué hacen:

Archivo	Qué configura
<code>.vscode/tasks.json</code>	La tarea Compilar C con gcc : con un archivo <code>.c</code> abierto, Ctrl + Shift + B ejecuta <code>gcc -Wall -g archivo.c -o archivo</code> .
<code>.vscode/settings.json</code>	Codificación UTF-8, sangría de 4 espacios, finales de línea de Linux y limpieza de espacios al guardar.

.vscode/tasks.json (incluido en la plantilla)

```
{
  // Tarea para compilar el archivo .c abierto con gcc (Ctrl + Shift + B)
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Compilar C con gcc",
      "type": "shell",
      "command": "gcc",
      "args": [
        "-Wall",
        "-g",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

5.6 Ajustes recomendados

1. **Guardado automático:** menú **Archivo** → **Guardado automático**. Evita compilar una versión vieja del archivo.
2. **Tamaño de letra:** Ctrl + coma → buscar **Font Size** → 16 (más cómodo para proyectar y leer).

6. Probar todo con un programa en C

1. En VS Code, abra la carpeta **C:\INA** (o la de su repositorio).
2. Cree el archivo **hola.c** y copie este programa:

```
hola.c
1 // hola.c - Prueba del entorno de la UA2
2 #include <stdio.h>           // biblioteca de entrada y salida: printf y scanf
3
4 int main(void) {
5     int edad;
6     // Muestra un mensaje -> Entorno listo para la UA2
7     printf("Entorno listo para la UA2\n");
8     // Pide y lee un número entero -> edad = 20
9     printf("Digite su edad: ");
10    scanf("%d", &edad);
11    // Muestra el dato leído -> Edad registrada: 20
12    printf("Edad registrada: %d\n", edad);
13    return 0;                // termina sin errores
14 }
```

N.º	Instrucción	¿Qué hace?	¿Cómo lo hace?	Resultado
2	#include <stdio.h>	Incluye la biblioteca de E/S.	El preprocesador inserta stdio.h.	printf y scanf disponibles.
5	int edad;	Declara un entero.	Reserva 4 bytes.	Existe edad.

N.º	Instrucción	¿Qué hace?	¿Cómo lo hace?	Resultado
7	<code>printf("Entorno listo...\n");</code>	Muestra un mensaje.	<code>\n</code> salta de línea.	Entorno listo para la UA2
9–10	<code>printf(...);</code> <code>scanf("%d", &edad);</code>	Pide y lee la edad.	<code>%d</code> lee un entero; <code>&</code> da la dirección.	edad = 20
12	<code>printf("Edad registrada: %d\n", edad);</code>	Muestra el dato.	<code>%d</code> se reemplaza por edad.	Edad registrada: 20
13	<code>return 0;</code>	Termina.	Devuelve 0 al sistema.	Fin exitoso.

Compile y ejecute de cualquiera de estas dos formas:

Comando	¿Qué hace?	Resultado esperado
<code>gcc -Wall hola.c -o hola</code>	Compila con todas las advertencias.	Sin mensajes (se crea hola.exe).
<code>./hola.exe</code>	Ejecuta el programa.	Pide la edad y la muestra.
<code>Ctrl + Shift + B</code>	Compila con la tarea de la plantilla.	La terminal indica que la tarea terminó sin errores.

Resultado esperado en la terminal (escribiendo 20)

```
Entorno listo para la UA2
Digite su edad: 20
Edad registrada: 20
```

¡Listo!

Si ve este resultado, su entorno está completo. Si aparece algún mensaje de error, consulte la sección 8.

7. Conectar con GitHub y el portafolio

1. Cree su cuenta en <https://github.com> (o inicie sesión) con su correo personal. Elija un nombre de usuario profesional.
2. Acepte la **invitación al repositorio** de su portafolio (llega por correo o en github.com/notifications).
3. En Git Bash, clone el repositorio dentro de C:\INA y entre a la carpeta (la URL la copia del botón **Code** en GitHub):

Comando	¿Qué hace?	Resultado esperado
<code>cd /c/INA</code>	Entra a la carpeta de trabajo.	El indicador muestra /c/INA
<code>git clone https://github.com/ORGANIZACION/REPOSITORIO.git</code>	Descarga su repositorio.	Cloning into ... done.
<code>cd REPOSITORIO</code>	Entra al repositorio.	El indicador muestra (main)

Comando	¿Qué hace?	Resultado esperado
<code>git config user.name "Nombre Apellido"</code>	Registra su nombre en este repositorio.	Sin salida visible.
<code>git config user.email "su-correo@ejemplo.com"</code>	Registra el correo de su cuenta de GitHub.	Sin salida visible.
<code>git config pull.rebase false</code>	Define cómo combinar cambios al hacer pull.	Sin salida visible.
<code>code .</code>	Abre el repositorio en VS Code.	VS Code se abre en la carpeta.

La primera vez que haga **git push**, Git Credential Manager abrirá el navegador para iniciar sesión en GitHub. En la sesión 15 se explica el flujo completo de trabajo con el portafolio.

En los equipos compartidos del laboratorio, al terminar la clase cierre su sesión de GitHub en el navegador y en el Administrador de credenciales de Windows (Credenciales de Windows → [git:https://github.com](https://github.com)).

8. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
"gcc no se reconoce como un comando interno o externo"	gcc no está en Path o la terminal se abrió antes de configurarlo.	Revisar la sección 3 y cerrar y abrir VS Code.
"bash: gcc: command not found" en Git Bash	Mismo problema de Path.	Igual que el anterior; reiniciar VS Code.
pacman: "could not lock database"	Hay otra terminal de MSYS2 instalando.	Cerrar las demás terminales de MSYS2 y repetir.
Ctrl + Shift + B no hace nada	Se abrió el archivo y no la carpeta, o falta .vscode/tasks.json.	Archivo → Abrir carpeta (sección 5.4).
El error "no se corrige" al compilar	El archivo no se guardó.	Ctrl + S o activar el guardado automático.
"Permission denied" al compilar	El programa anterior sigue en ejecución.	Cerrarlo (Ctrl + C en la terminal) y compilar de nuevo.
El antivirus borra o bloquea el .exe	Detección falsa de un ejecutable nuevo.	Avisar a la persona facilitadora para agregar una exclusión a la carpeta C:\INA.
Las tildes se ven como símbolos raros	Codificación de la consola.	En printf, escribir por ahora sin tildes ni ñ.
El mensaje de printf aparece después de escribir el dato	La terminal no muestra la salida hasta un salto de línea (fuera de VS Code).	Ejecutar en la terminal de VS Code o agregar fflush(stdout); antes de scanf.
git push pide usuario y contraseña	Falta iniciar sesión con Git Credential Manager.	Usar la opción de iniciar sesión con el navegador.

9. Lista de verificación final

Verificación	Sí	No
gcc --version muestra "Built by MSYS2 project".		
git --version muestra la versión de Git para Windows.		
VS Code tiene instalada la extensión C/C++ de Microsoft.		
La terminal predeterminada de VS Code es Git Bash.		
hola.c compila sin mensajes y ./hola.exe funciona.		
Ctrl + Shift + B compila el archivo abierto.		
El repositorio del portafolio está clonado en C:\INA.		
git config muestra su nombre y su correo en el repositorio.		

Anexo · Para la persona facilitadora: preparación del laboratorio

- Instale MSYS2 en **C:\msys64** en cada equipo y agregue **C:\msys64\ucrt64\bin** al Path del **sistema** (Variables del sistema), para que funcione con cualquier cuenta de Windows.
- Verifique en los 16 equipos: **gcc --version**, **git --version** y **code --version** en una terminal nueva.
- Solicite al soporte técnico una exclusión del antivirus para **C:\INA** si los .exe son bloqueados.
- Si no hay internet suficiente para todos, instale primero en un equipo y comparta los instaladores descargados (MSYS2, Git y VS Code) en una memoria USB; el comando pacman sí requiere conexión.
- Pida a cada persona completar la lista de verificación de la sección 9 antes de la sesión 16.

Fuentes oficiales: Visual Studio Code, "Using GCC with MinGW" (code.visualstudio.com/docs/cpp/config-mingw); MSYS2 (www.msys2.org); Git para Windows (git-scm.com).